



Recursion Algorithms

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Contents

- Introduction to Recursion
- Types of Recursion
- Direct Recursion
- Indirect Recursion
- Tail Recursion
- Head Recursion
- Why Use Recursion?
- When to Avoid Recursion
- Common Examples of Recursion

Introduction to Recursion

Recursion is a programming technique where a function calls itself.

Solve a problem by breaking it down into smaller subproblems of the same type.

```
void printNumbers(int n) {  
  
    cout << n << " "<<endl;    // Print the number  
    printNumbers(n - 1); // Recursive call with a smaller value  
}
```

Introduction to Recursion

Base Case: Condition to stop recursion

```
void printNumbers(int n) {  
    if (n == 0) // Base case  
        return;  
  
    cout << n << " "<<endl; // Print the number  
    printNumbers(n - 1); // Recursive call with a smaller value  
}
```

Types of Recursion

1.Direct Recursion

2.Indirect Recursion

3.Tail Recursion

4.Head Recursion

5.Tree Recursion

6.Nested Recursion

Direct Recursion

Direct Recursion: A function directly calls itself.

```
int factorial(int n) {  
    if (n <= 1)    // Base case  
        return 1;  
    return n * factorial(n - 1);    // Direct recursive call  
}
```

Indirect Recursion

Indirect Recursion: A function calls another function, which in turn calls the original function.

```
void functionA(int n)
{ if (n > 0) {
cout << "A: " << n << endl;
functionB(n - 1); // Calls functionB
} }

void functionB(int n)
{ if (n > 0) {
cout << "B: " << n << endl;
functionA(n / 2); // Calls functionA
} }
```

Tail Recursion

Tail Recursion: The recursive call is the last operation before returning the final value.

```
void tailFactorial(int n, int result)
{
    if (n == 0)
    {
        cout << "Factorial: " << result << endl;
        return;
    }
    tailFactorial(n - 1, result * n); // Recursive call is the last operation
}
```

Head Recursion

Head Recursion: The recursive call is the first operation before any other computation.

```
void headRecursion(int n)

{

if (n == 0)

return;

headRecursion(n - 1);  // Recursive call happens first

cout << n << endl;  // Processing after recursion

}
```

Why Use Recursion?

1. Simplifies complex problems
2. Reduces code length
3. Useful for problems with a naturally recursive structure (e.g., Trees, Graphs)
4. Helps in backtracking-based algorithms
5. Reuse code

When to Avoid Recursion

1. When performance is critical
2. When memory usage is a concern
3. When iterative solutions are simpler and more efficient

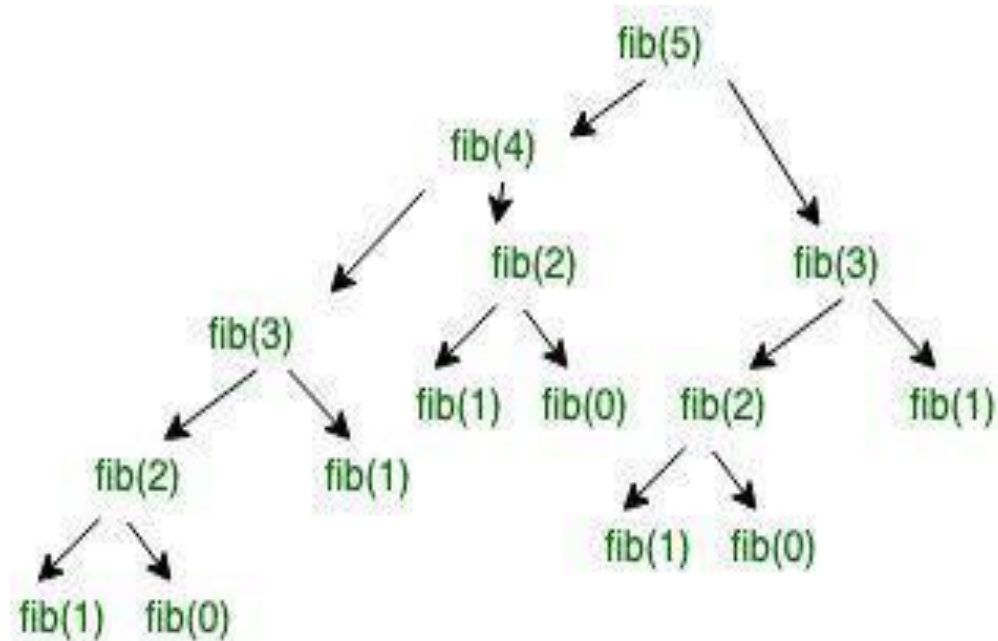
Example: Fibonacci with memoization

Common Examples of Recursion

Recursion Function for fibonacci

```
int fib(int n){  
    // Stop condition  
    if (n == 0)  
        return 0;  
    if (n == 1)  
        return 1;  
    // Recursion function  
    else{  
        return (fib(n - 1) + fib(n - 2));  
    }  
}
```

Recursion Tree for fibonacci



Common Examples of Recursion

Recursion Function for factorial

```
int fact(int n)

{

// BASE CONDITION

if (n == 0)

return 1;

return n * fact(n - 1);

}
```

Assessment

Draw a recursion tree for calculating the factorial of 6 ($F(6)$).

Thanks

